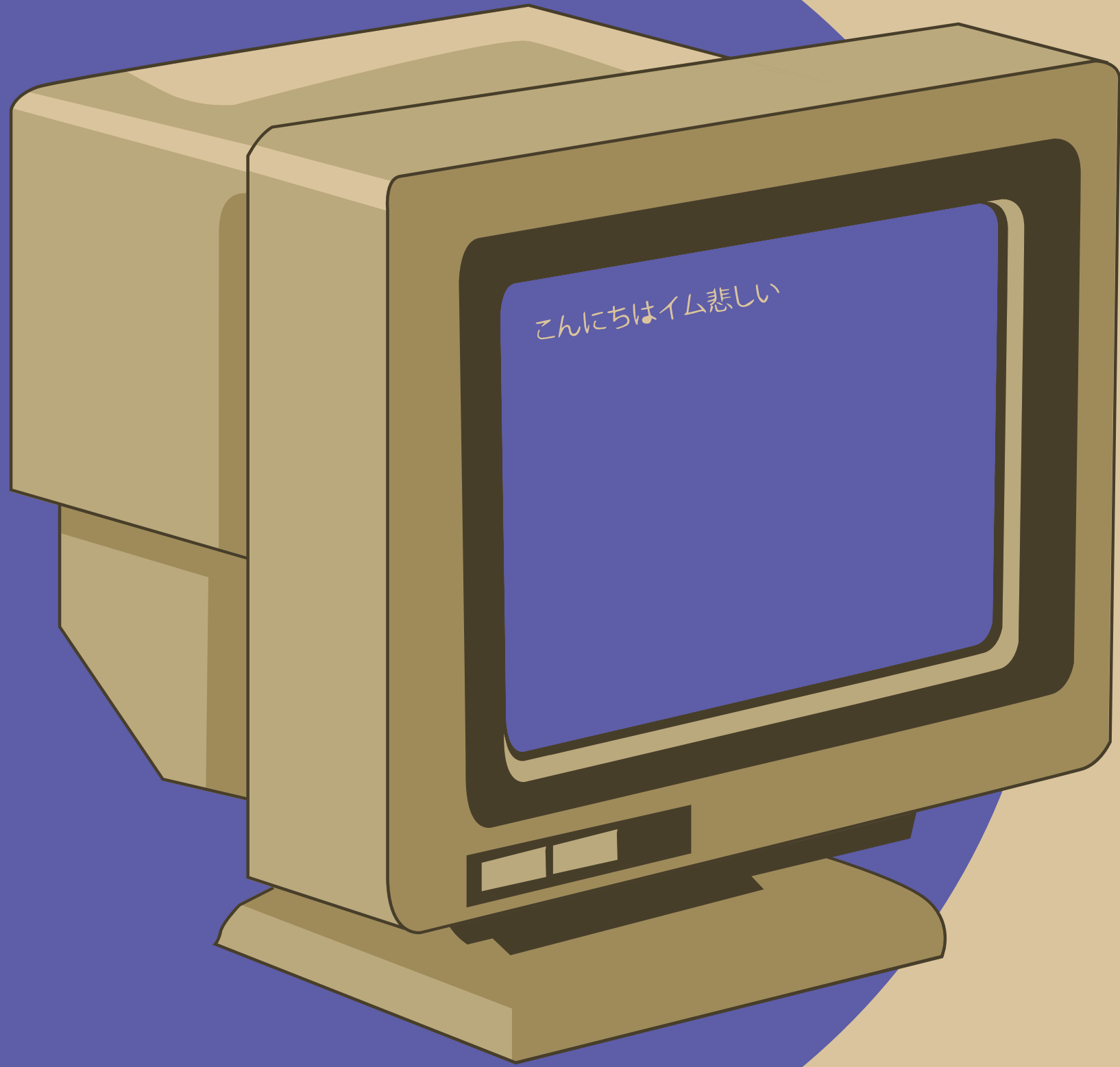




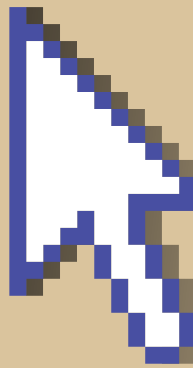
GROUP 10

16-CHANNEL PARALLEL SORTING NETWORK (BITONIC SORTER)

FUNDAMENTAL OF DIGITAL SYSTEM FINAL PROJECT
REPORT



KELOMPOK10_FINPRO_PSD

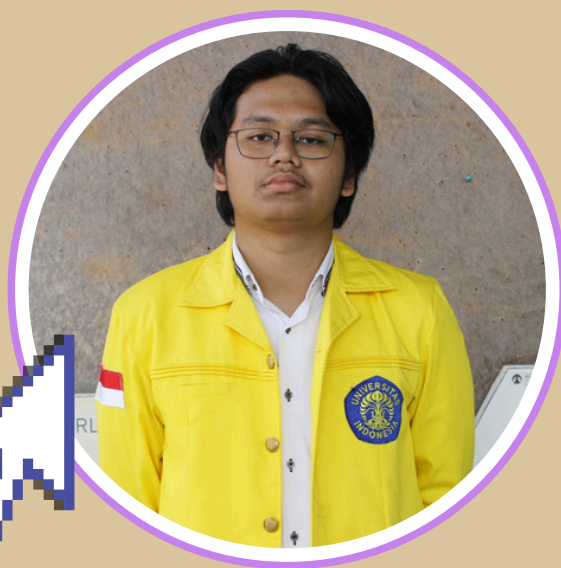




GROUP 10



ANGGOTA KELOMPOK



**CLEMENTHEO
BENAYA R. S.**



**HAIDAR RAFIF
RADITHYA**



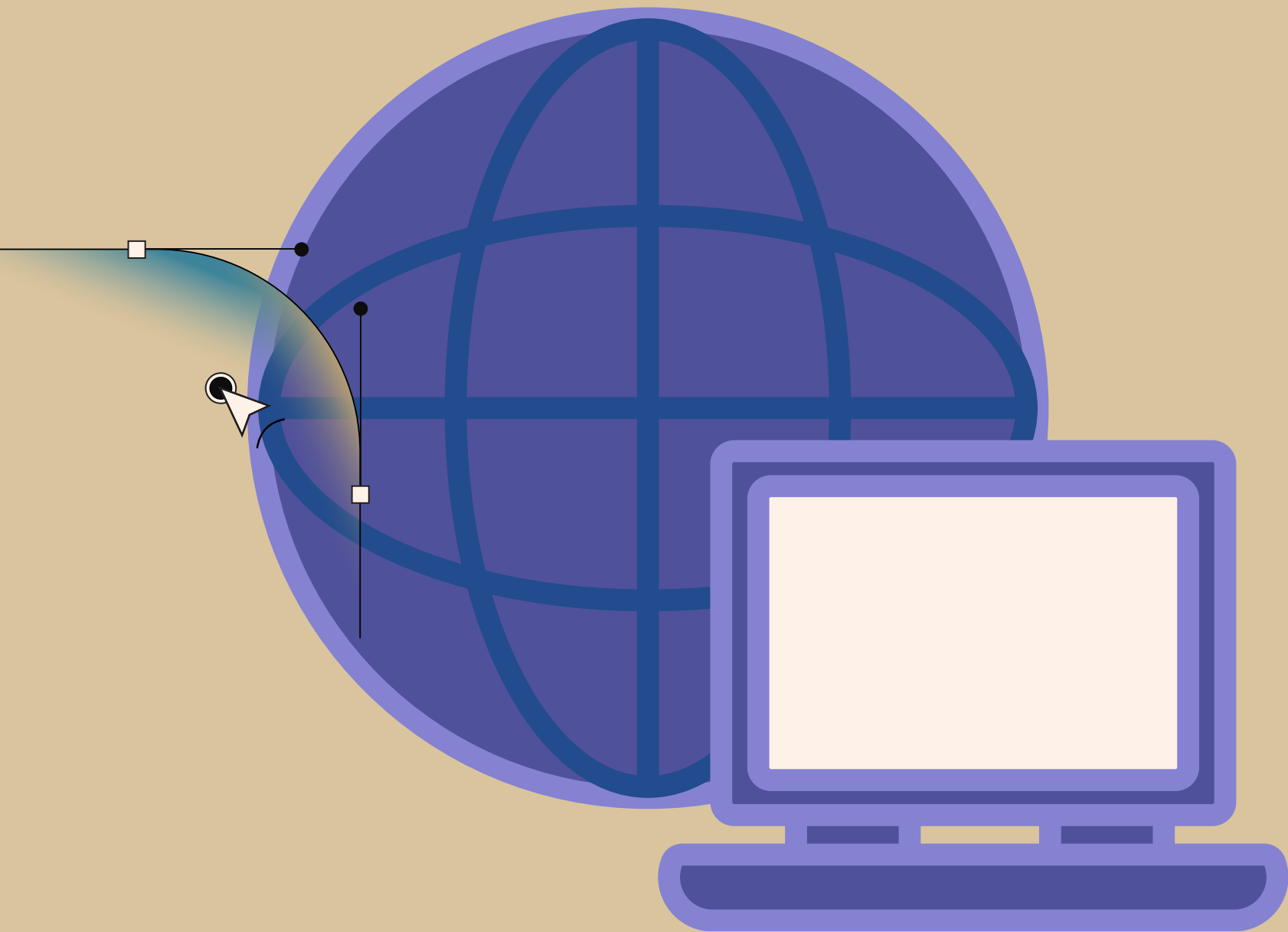
**RAJA AVICENNA AL-
KINDI V.**



**MUHAMMAD ZAKI AL
KHAIRI**



GROUP 10



BACKGROUND

PERKEMBANGAN SISTEM TERTANAM DAN IOT MENUNTUT PEMROSESAN DATA BERKECEPATAN TINGGI UNTUK APLIKASI KRITIS SEPERTI RADAR, NAVIGASI OTONOM, DAN INSTRUMENTASI MEDIS, DI MANA PROSES SORTING DIPERLUKAN UNTUK VALIDASI DATA SECARA REAL-TIME. NAMUN, ALGORITMA SORTING PADA CPU KONVENSIONAL MEMILIKI KETERBATASAN KARENA BEKERJA SECARA SEKUENSIAL, SEHINGGA MENIMBULKAN LATENSI TINGGI DAN RISIKO BOTTLENECK KETIKA VOLUME DATA MENINGKAT. UNTUK MENGATASI HAL TERSEBUT, DIPERLUKAN SOLUSI BERBASIS PERANGKAT KERAS YANG MAMPU MELAKUKAN PEMROSESAN PARALEL. FPGA MENJADI PILIHAN IDEAL KARENA DAPAT DIKONFIGURASI SECARA KUSTOM PADA LEVEL GERBANG LOGIKA, MEMUNGKINKAN OPERASI SORTING DILAKUKAN SERENTAK. OLEH KARENA ITU, PROYEK INI MERANCANG "16-CHANNEL PARALLEL SORTING NETWORK" BERBASIS BITONIC SORT SEBAGAI AKSELERATOR PERANGKAT KERAS UNTUK MENINGKATKAN THROUGHPUT DAN MENGURANGI BEBAN KOMPUTASI PROSESOR UTAMA.





GROUP 10

PROJECT DESCRIPTION

Proyek akhir “16-Channel Parallel Sorting Network” ini merancang sebuah akselerator perangkat keras berbasis FPGA untuk mengurutkan 16 data integer secara paralel menggunakan algoritma Bitonic Sort. Berbeda dari pengurutan berbasis CPU yang bersifat iteratif, Sorting Network memungkinkan data melewati rangkaian komparator fisik dengan latensi deterministik dan throughput tinggi. Sistem dibangun menggunakan blok Compare-and-Swap (CAS) yang disusun bertingkat melalui pendekatan struktural dan generate statement untuk mempermudah pembentukan jaringan komparator yang repetitif, sementara prosedur digunakan untuk menjaga modularitas logika pertukaran data. Selain datapath utama, sistem dilengkapi Control Unit berbasis FSM yang mengatur aliran data dari input hingga output. Tipe data dan konstanta disusun dalam package terpisah agar struktur proyek rapi, dan fungsionalitas keseluruhan divalidasi melalui testbench otomatis yang menguji dan memverifikasi kebenaran hasil pengurutan.





GROUP 10

OBJECTIVES



SEBAGAI PEMENUHAN NILAI DALAM MATA KULIAH PERANCANGAN SISTEM DIGITAL.



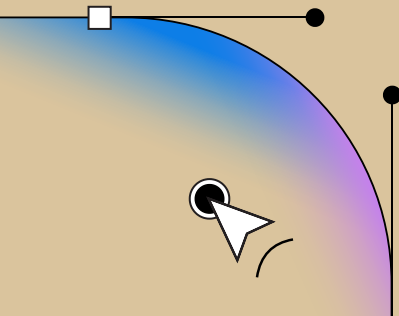
MENDEMONSTRASIKAN PENGGUNAAN LOOPING CONSTRUCT UNTUK MENYUSUN RATUSAN GERBANG LOGIKA SECARA OTOMATIS DAN EFISIEN TANPA MENULIS ULANG KODE MANUAL.



MENGIMPLEMENTASIKAN PEMROGRAMAN VHDL YANG BERSIFAT SOLUTIF.



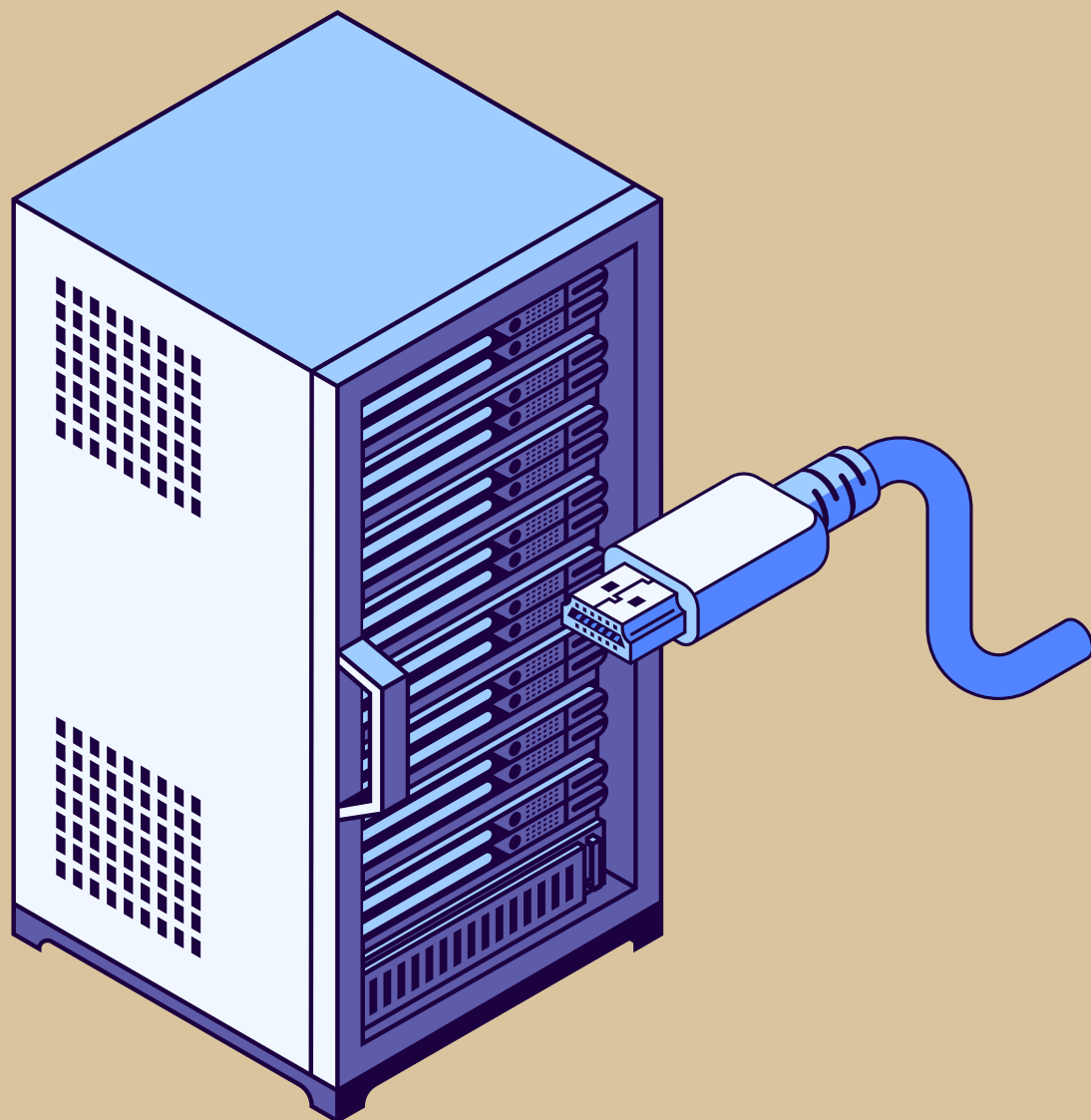
MENGHASILKAN SISTEM DENGAN WAKTU EKSEKUSI YANG TETAP DAN PASTI, YANG MERUPAKAN SYARAT UTAMA DALAM SISTEM DIGITAL REAL-TIME BERKINERJA TINGGI.





GROUP 10

TOOLS



Visual Studio Code



Github



Quartus Prime



ModelSim





BITONIC SORTER

SORTER PACKAGE (sorter_pkg.vhd)

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use IEEE.MATH_REAL.ALL;
5  use work.sorter_pkg.all;
6
7  entity bitonic_net is
8      Port (
9          i_data : in  t_data_array;
10         o_data : out t_data_array
11     );
12 end bitonic_net;
13
14 architecture Structural of bitonic_net is
15
16     component cas_unit is
17         Port (
18             i_val_a : in  SIGNED(DATA_WIDTH-1 downto 0);
19             i_val_b : in  SIGNED(DATA_WIDTH-1 downto 0);
20             o_min  : out SIGNED(DATA_WIDTH-1 downto 0);
21             o_max  : out SIGNED(DATA_WIDTH-1 downto 0)
22         );
23     end component;
24
25     constant NUM_STAGES : integer := 10;
26
27     type t_stage_array is array (0 to NUM_STAGES) of t_data_array;
28     signal stage : t_stage_array;
29
30 begin
31
32     stage(0) <= i_data;
33
34     gen_stage1: for i in 0 to 7 generate
35         cas1: cas_unit port map(
36             i_val_a => stage(0)(2*i),
37             i_val_b => stage(0)(2*i+1),
38             o_min  => stage(1)(2*i),
39             o_max  => stage(1)(2*i+1)
40         );
41     end generate;
42
```

```
43     gen_stage2: for i in 0 to 3 generate
44         cas2a: cas_unit port map(
45             i_val_a => stage(1)(4*i),
46             i_val_b => stage(1)(4*i+3),
47             o_min  => stage(2)(4*i),
48             o_max  => stage(2)(4*i+3)
49         );
50         cas2b: cas_unit port map(
51             i_val_a => stage(1)(4*i+1),
52             i_val_b => stage(1)(4*i+2),
53             o_min  => stage(2)(4*i+1),
54             o_max  => stage(2)(4*i+2)
55         );
56     end generate;
57
58     gen_stage3: for i in 0 to 3 generate
59         cas3: cas_unit port map(
60             i_val_a => stage(2)(4*i),
61             i_val_b => stage(2)(4*i+1),
62             o_min  => stage(3)(4*i),
63             o_max  => stage(3)(4*i+1)
64         );
65         cas3b: cas_unit port map(
66             i_val_a => stage(2)(4*i+2),
67             i_val_b => stage(2)(4*i+3),
68             o_min  => stage(3)(4*i+2),
69             o_max  => stage(3)(4*i+3)
70         );
71     end generate;
72
```

```
73     gen_stage4: for i in 0 to 1 generate
74         cas4a: cas_unit port map(
75             i_val_a => stage(3)(8*i),
76             i_val_b => stage(3)(8*i+7),
77             o_min  => stage(4)(8*i),
78             o_max  => stage(4)(8*i+7)
79         );
80         cas4b: cas_unit port map(
81             i_val_a => stage(3)(8*i+1),
82             i_val_b => stage(3)(8*i+6),
83             o_min  => stage(4)(8*i+1),
84             o_max  => stage(4)(8*i+6)
85         );
86         cas4c: cas_unit port map(
87             i_val_a => stage(3)(8*i+2),
88             i_val_b => stage(3)(8*i+5),
89             o_min  => stage(4)(8*i+2),
90             o_max  => stage(4)(8*i+5)
91         );
92         cas4d: cas_unit port map(
93             i_val_a => stage(3)(8*i+3),
94             i_val_b => stage(3)(8*i+4),
95             o_min  => stage(4)(8*i+3),
96             o_max  => stage(4)(8*i+4)
97         );
98     end generate;
99
100     gen_stage5: for i in 0 to 1 generate
101         gen_stage5_inner: for j in 0 to 1 generate
102             cas5a: cas_unit port map(
103                 i_val_a => stage(4)(8*i+4*j),
104                 i_val_b => stage(4)(8*i+4*j+3),
105                 o_min  => stage(5)(8*i+4*j),
106                 o_max  => stage(5)(8*i+4*j+3)
107             );
108             cas5b: cas_unit port map(
109                 i_val_a => stage(4)(8*i+4*j+1),
110                 i_val_b => stage(4)(8*i+4*j+2),
111                 o_min  => stage(5)(8*i+4*j+1),
112                 o_max  => stage(5)(8*i+4*j+2)
113             );
114         end generate;
115     end generate;
116
```

```
117     gen_stage6: for i in 0 to 1 generate
118         gen_stage6_inner: for j in 0 to 3 generate
119             cas6: cas_unit port map(
120                 i_val_a => stage(5)(8*i+2*j),
121                 i_val_b => stage(5)(8*i+2*j+1),
122                 o_min  => stage(6)(8*i+2*j),
123                 o_max  => stage(6)(8*i+2*j+1)
124             );
125         end generate;
126     end generate;
127
128     cas7_0: cas_unit port map(stage(6)(0), stage(6)(15), stage(7)(0), stage(7)(15));
129     cas7_1: cas_unit port map(stage(6)(1), stage(6)(14), stage(7)(1), stage(7)(14));
130     cas7_2: cas_unit port map(stage(6)(2), stage(6)(13), stage(7)(2), stage(7)(13));
131     cas7_3: cas_unit port map(stage(6)(3), stage(6)(12), stage(7)(3), stage(7)(12));
132     cas7_4: cas_unit port map(stage(6)(4), stage(6)(11), stage(7)(4), stage(7)(11));
133     cas7_5: cas_unit port map(stage(6)(5), stage(6)(10), stage(7)(5), stage(7)(10));
134     cas7_6: cas_unit port map(stage(6)(6), stage(6)(9), stage(7)(6), stage(7)(9));
135     cas7_7: cas_unit port map(stage(6)(7), stage(6)(8), stage(7)(7), stage(7)(8));
136
137     gen_stage8: for i in 0 to 1 generate
138         cas8a: cas_unit port map(stage(7)(8*i), stage(7)(8*i+7), stage(8)(8*i), stage(8)(8*i+7));
139         cas8b: cas_unit port map(stage(7)(8*i+1), stage(7)(8*i+6), stage(8)(8*i+1), stage(8)(8*i+6));
140         cas8c: cas_unit port map(stage(7)(8*i+2), stage(7)(8*i+5), stage(8)(8*i+2), stage(8)(8*i+5));
141         cas8d: cas_unit port map(stage(7)(8*i+3), stage(7)(8*i+4), stage(8)(8*i+3), stage(8)(8*i+4));
142     end generate;
143
144     gen_stage9: for i in 0 to 3 generate
145         cas9a: cas_unit port map(stage(8)(4*i), stage(8)(4*i+3), stage(9)(4*i), stage(9)(4*i+3));
146         cas9b: cas_unit port map(stage(8)(4*i+1), stage(8)(4*i+2), stage(9)(4*i+1), stage(9)(4*i+2));
147     end generate;
148
149     gen_stage10: for i in 0 to 7 generate
150         cas10: cas_unit port map(
151             i_val_a => stage(9)(2*i),
152             i_val_b => stage(9)(2*i+1),
153             o_min  => stage(10)(2*i),
154             o_max  => stage(10)(2*i+1)
155         );
156     end generate;
157
158     o_data <= stage(10);
159
160 end Structural;
```

File ini berfungsi sebagai library global yang menstandarisasi tipe data di seluruh sistem. Di dalamnya didefinisikan konstanta DATA_WIDTH sebesar 16 bit dan ARRAY_SIZE sebesar 16 elemen, serta tipe data kustom t_data_array. Modul ini juga membuat pure function is_sorted dan find_min serta impure function log_array yang digunakan untuk memverifikasi kebenaran data dan keperluan logging simulasi



COMPARE & SWAP UNIT (cas_unit.vhd)

Unit ini merupakan unit komputasi dasar dari sistem yang kita buat. Unit cas_unit memiliki port input i_val_a dan i_val_b bertipe SIGNED. Kita menggunakan fungsi internal get_min dan get_max untuk membandingkan kedua nilai input tersebut. Output o_min akan selalu mengeluarkan nilai terkecil dan output o_max akan selalu mengeluarkan nilai terbesar, sehingga memastikan data yang keluar dalam kondisi terurut secara lokal.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4  use work.sorter_pkg.all;
5
6  entity cas_unit is
7      Port (
8          i_val_a : in  SIGNED(DATA_WIDTH-1 downto 0);
9          i_val_b : in  SIGNED(DATA_WIDTH-1 downto 0);
10         o_min  : out SIGNED(DATA_WIDTH-1 downto 0);
11         o_max  : out SIGNED(DATA_WIDTH-1 downto 0)
12     );
13 end cas_unit;
14
15 architecture Behavioral of cas_unit is
16
17     function get_min(val1, val2: SIGNED) return SIGNED is
18     begin
19         if val1 <= val2 then
20             return val1;
21         else
22             return val2;
23         end if;
24     end function get_min;
25
26     function get_max(val1, val2 : SIGNED) return SIGNED is
27     begin
28         if val1 >= val2 then
29             return val1;
30         else
31             return val2;
32         end if;
33     end function get_max;
34
35
36     begin
37         o_min <= get_min(i_val_a, i_val_b);
38         o_max <= get_max(i_val_a, i_val_b);
39
40     end Behavioral;
```



BITONIC SORTER

BITONIC NETWORK (bitonic_net.vhd)

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use IEEE.MATH_REAL.ALL;
5  use work.sorter_pkg.all;
6
7  entity bitonic_net is
8      Port (
9          i_data : in  t_data_array;
10         o_data : out t_data_array
11     );
12 end bitonic_net;
13
14 architecture Structural of bitonic_net is
15
16     component cas_unit is
17         Port (
18             i_val_a : in  SIGNED(DATA_WIDTH-1 downto 0);
19             i_val_b : in  SIGNED(DATA_WIDTH-1 downto 0);
20             o_min  : out SIGNED(DATA_WIDTH-1 downto 0);
21             o_max  : out SIGNED(DATA_WIDTH-1 downto 0)
22         );
23     end component;
24
25     constant NUM_STAGES : integer := 10;
26
27     type t_stage_array is array (0 to NUM_STAGES) of t_data_array;
28     signal stage : t_stage_array;
29
30 begin
31
32     stage(0) <= i_data;
33
```

```
34     gen_stage1: for i in 0 to 7 generate
35         cas1: cas_unit port map(
36             i_val_a => stage(0)(2*i),
37             i_val_b => stage(0)(2*i+1),
38             o_min  => stage(1)(2*i),
39             o_max  => stage(1)(2*i+1)
40         );
41     end generate;
42
43     gen_stage2: for i in 0 to 3 generate
44         cas2a: cas_unit port map(
45             i_val_a => stage(1)(4*i),
46             i_val_b => stage(1)(4*i+3),
47             o_min  => stage(2)(4*i),
48             o_max  => stage(2)(4*i+3)
49         );
50         cas2b: cas_unit port map(
51             i_val_a => stage(1)(4*i+1),
52             i_val_b => stage(1)(4*i+2),
53             o_min  => stage(2)(4*i+1),
54             o_max  => stage(2)(4*i+2)
55         );
56     end generate;
57
58     gen_stage3: for i in 0 to 3 generate
59         cas3: cas_unit port map(
60             i_val_a => stage(2)(4*i),
61             i_val_b => stage(2)(4*i+1),
62             o_min  => stage(3)(4*i),
63             o_max  => stage(3)(4*i+1)
64         );
65         cas3b: cas_unit port map(
66             i_val_a => stage(2)(4*i+2),
67             i_val_b => stage(2)(4*i+3),
68             o_min  => stage(3)(4*i+2),
69             o_max  => stage(3)(4*i+3)
70         );
71     end generate;
72
```

```
73     gen_stage4: for i in 0 to 1 generate
74         cas4a: cas_unit port map(
75             i_val_a => stage(3)(8*i),
76             i_val_b => stage(3)(8*i+7),
77             o_min  => stage(4)(8*i),
78             o_max  => stage(4)(8*i+7)
79         );
80         cas4b: cas_unit port map(
81             i_val_a => stage(3)(8*i+1),
82             i_val_b => stage(3)(8*i+6),
83             o_min  => stage(4)(8*i+1),
84             o_max  => stage(4)(8*i+6)
85         );
86         cas4c: cas_unit port map(
87             i_val_a => stage(3)(8*i+2),
88             i_val_b => stage(3)(8*i+5),
89             o_min  => stage(4)(8*i+2),
90             o_max  => stage(4)(8*i+5)
91         );
92         cas4d: cas_unit port map(
93             i_val_a => stage(3)(8*i+3),
94             i_val_b => stage(3)(8*i+4),
95             o_min  => stage(4)(8*i+3),
96             o_max  => stage(4)(8*i+4)
97         );
98     end generate;
99
100     gen_stage5: for i in 0 to 1 generate
101         gen_stage5_inner: for j in 0 to 1 generate
102             cas5a: cas_unit port map(
103                 i_val_a => stage(4)(8*i+4*j),
104                 i_val_b => stage(4)(8*i+4*j+3),
105                 o_min  => stage(5)(8*i+4*j),
106                 o_max  => stage(5)(8*i+4*j+3)
107             );
108             cas5b: cas_unit port map(
109                 i_val_a => stage(4)(8*i+4*j+1),
110                 i_val_b => stage(4)(8*i+4*j+2),
111                 o_min  => stage(5)(8*i+4*j+1),
112                 o_max  => stage(5)(8*i+4*j+2)
113             );
114         end generate;
115     end generate;
116
```

```
117     gen_stage6: for i in 0 to 1 generate
118         gen_stage6_inner: for j in 0 to 3 generate
119             cas6: cas_unit port map(
120                 i_val_a => stage(5)(8*i+2*j),
121                 i_val_b => stage(5)(8*i+2*j+1),
122                 o_min  => stage(6)(8*i+2*j),
123                 o_max  => stage(6)(8*i+2*j+1)
124             );
125         end generate;
126     end generate;
127
128     cas7_0: cas_unit port map(stage(6)(0), stage(6)(15), stage(7)(0), stage(7)(15));
129     cas7_1: cas_unit port map(stage(6)(1), stage(6)(14), stage(7)(1), stage(7)(14));
130     cas7_2: cas_unit port map(stage(6)(2), stage(6)(13), stage(7)(2), stage(7)(13));
131     cas7_3: cas_unit port map(stage(6)(3), stage(6)(12), stage(7)(3), stage(7)(12));
132     cas7_4: cas_unit port map(stage(6)(4), stage(6)(11), stage(7)(4), stage(7)(11));
133     cas7_5: cas_unit port map(stage(6)(5), stage(6)(10), stage(7)(5), stage(7)(10));
134     cas7_6: cas_unit port map(stage(6)(6), stage(6)(9), stage(7)(6), stage(7)(9));
135     cas7_7: cas_unit port map(stage(6)(7), stage(6)(8), stage(7)(7), stage(7)(8));
136
137     gen_stage8: for i in 0 to 1 generate
138         cas8a: cas_unit port map(stage(7)(8*i), stage(7)(8*i+7), stage(8)(8*i), stage(8)(8*i+7));
139         cas8b: cas_unit port map(stage(7)(8*i+1), stage(7)(8*i+6), stage(8)(8*i+1), stage(8)(8*i+6));
140         cas8c: cas_unit port map(stage(7)(8*i+2), stage(7)(8*i+5), stage(8)(8*i+2), stage(8)(8*i+5));
141         cas8d: cas_unit port map(stage(7)(8*i+3), stage(7)(8*i+4), stage(8)(8*i+3), stage(8)(8*i+4));
142     end generate;
143
144     gen_stage9: for i in 0 to 3 generate
145         cas9a: cas_unit port map(stage(8)(4*i), stage(8)(4*i+3), stage(9)(4*i), stage(9)(4*i+3));
146         cas9b: cas_unit port map(stage(8)(4*i+1), stage(8)(4*i+2), stage(9)(4*i+1), stage(9)(4*i+2));
147     end generate;
148
149     gen_stage10: for i in 0 to 7 generate
150         cas10: cas_unit port map(
151             i_val_a => stage(9)(2*i),
152             i_val_b => stage(9)(2*i+1),
153             o_min  => stage(10)(2*i),
154             o_max  => stage(10)(2*i+1)
155         );
156     end generate;
157
158     o_data <= stage(10);
159
160 end Structural;
```

Merupakan entitas utama yang mengimplementasikan algoritma sorting menggunakan gaya pemrograman structural. File ini menyusun puluhan unit cas_unit ke dalam 10 stage menggunakan konstruksi GENERATE statement. Sinyal internal array digunakan untuk menghubungkan output dari satu stage ke input dari stage berikutnya secara besar-besaran dan bersamaan. Akibatnya, seluruh array data input diproses secara simultan dan menghasilkan output terurut di port o_data.



TESTBENCH



TESTBENCH

(tb_sorter.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.MATH_REAL.ALL;
use work.sorter_pkg.all;

end tb_sorter;

architecture Behavioral of tb_sorter is

    component top_sorter is
        Port (
            i_clk      : in  std_logic;
            i_rst      : in  std_logic;
            i_start     : in  std_logic;
            i_raw_data  : in  t_data_array;
            o_sorted_data : out t_data_array;
            o_done       : out std_logic
        );
    end component;

    constant CLK_PERIOD : time := 10 ns;

    signal clk      : std_logic := '0';
    signal rst      : std_logic := '0';
    signal start    : std_logic := '0';
    signal raw_data : t_data_array := (others => (others => '0'));
    signal sorted_data : t_data_array;
    signal done      : std_logic;
    signal sim_done  : boolean := false;

begin
```

```
    for i in 0 to ARRAY_SIZE-1 loop
        uniform(seed1, seed2, rand_val);
        int_val := integer(rand_val * 1000.0 - 500.0);
        raw_data(i) <= to_signed(int_val, DATA_WIDTH);
    end loop;

    log_status := log_array(raw_data, "Test1-Input");
    min_val := find_min(raw_data);
    report "Min: " & integer'image(to_integer(min_val)) severity note;

    wait for CLK_PERIOD;
    start <= '1';
    wait for CLK_PERIOD;
    start <= '0';

    wait until done = '1';
    wait for CLK_PERIOD;

    report "Checking sorted output..." severity note;
    test_passed := true;

    for i in 0 to ARRAY_SIZE-1 loop
        report "Output[" & integer'image(i) & "] = " &
            integer'image(to_integer(sorted_data(i))) severity note;
    end loop;

    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
            report "SORTING FAILED at index " & integer'image(i) &
                ": " & integer'image(to_integer(sorted_data(i))) &
                " > " & integer'image(to_integer(sorted_data(i+1)))
                severity error;
```

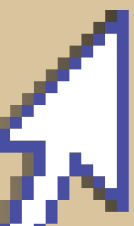
```
begin

    UUT: top_sorter
        port map (
            i_clk      => clk,
            i_rst      => rst,
            i_start     => start,
            i_raw_data  => raw_data,
            o_sorted_data => sorted_data,
            o_done       => done
        );

    clk_process: process
    begin
        while not sim_done loop
            clk <= '0';
            wait for CLK_PERIOD/2;
            clk <= '1';
            wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process clk_process;

    stimulus_process: process
    begin
        variable seed1, seed2 : positive := 1;
        variable rand_val     : real;
        variable int_val      : integer;
        variable test_passed  : boolean;
        variable log_status   : integer;
        variable min_val      : SIGNED(DATA_WIDTH-1 downto 0);

        -- Test 1: Reset test
        report "=====" severity note;
        report "Starting Testbench for Bitonic Sorter" severity note;
        report "=====" severity note;
```





TESTBENCH



TESTBENCH

(tb_sorter.vhd)

```
        if sorted_data(i) > sorted_data(i+1) then
            test_passed := false;
        end if;
    end loop;

    if test_passed then
        report "*** TEST 1 PASSED ***" severity note;
    else
        report "*** TEST 1 FAILED ***" severity error;
    end if;

    wait for CLK_PERIOD * 5;

    report "===== " severity note;
    report "Test 2: Testing with already sorted data..." severity note;

    for i in 0 to ARRAY_SIZE-1 loop
        raw_data(i) <= to_signed(i * 10, DATA_WIDTH);
    end loop;

    start <= '1';
    wait for CLK_PERIOD;
    start <= '0';

    wait until done = '1';
    wait for CLK_PERIOD;

    test_passed := true;
    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
            report "SORTING FAILED in Test 2"
            severity error;
```

```
        if sorted_data(i) > sorted_data(i+1) then
            test_passed := false;
        end if;
    end loop;

    if test_passed then
        report "*** TEST 2 PASSED ***" severity note;
    else
        report "*** TEST 2 FAILED ***" severity error;
    end if;

    wait for CLK_PERIOD * 5;

    report "===== " severity note;
    report "Test 3: Testing with reverse sorted data..." severity note;

    for i in 0 to ARRAY_SIZE-1 loop
        raw_data(i) <= to_signed((ARRAY_SIZE-1-i) * 10, DATA_WIDTH);
    end loop;

    start <= '1';
    wait for CLK_PERIOD;
    start <= '0';

    wait until done = '1';
    wait for CLK_PERIOD;

    test_passed := true;
    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
```

```
    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
            report "SORTING FAILED in Test 3"
            severity error;

        if sorted_data(i) > sorted_data(i+1) then
            test_passed := false;
        end if;
    end loop;

    if test_passed then
        report "*** TEST 3 PASSED ***" severity note;
    else
        report "*** TEST 3 FAILED ***" severity error;
    end if;

    wait for CLK_PERIOD * 5;
    report "===== " severity note;
    report "Test 4: Testing with all identical values..." severity note;

    for i in 0 to ARRAY_SIZE-1 loop
        raw_data(i) <= to_signed(42, DATA_WIDTH);
    end loop;

    start <= '1';
    wait for CLK_PERIOD;
    start <= '0';

    wait until done = '1';
    wait for CLK_PERIOD;

    test_passed := true;
    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
```



TESTBENCH

TESTBENCH

(tb_sorter.vhd)

```
        if sorted_data(i) > sorted_data(i+1) then
            test_passed := false;
        end if;
    end loop;

    if test_passed then
        report "*** TEST 4 PASSED ***" severity note;
    else
        report "*** TEST 4 FAILED ***" severity error;
    end if;

    report "===== " severity note;
    report "Test 5: Testing with negative numbers..." severity note;

    for i in 0 to ARRAY_SIZE-1 loop
        uniform(seed1, seed2, rand_val);
        int_val := integer(rand_val * 2000.0 - 1000.0); -- Range: -1000 to 1000
        raw_data(i) <= to_signed(int_val, DATA_WIDTH);
    end loop;

    start <= '1';
    wait for CLK_PERIOD;
    start <= '0';

    wait until done = '1';
    wait for CLK_PERIOD;

    test_passed := true;
    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
            report "SORTING FAILED in Test 5"
```

```
        test_passed := true;
    for i in 0 to ARRAY_SIZE-2 loop
        assert (sorted_data(i) <= sorted_data(i+1))
            report "SORTING FAILED in Test 5"
            severity error;

        if sorted_data(i) > sorted_data(i+1) then
            test_passed := false;
        end if;
    end loop;

    if test_passed then
        report "*** TEST 5 PASSED ***" severity note;
    else
        report "*** TEST 5 FAILED ***" severity error;
    end if;

    wait for CLK_PERIOD * 5;
    report "===== " severity note;
    report "ALL TESTS COMPLETED" severity note;
    report "===== " severity note;

    sim_done <= true;
    wait;
end process stimulus_process;

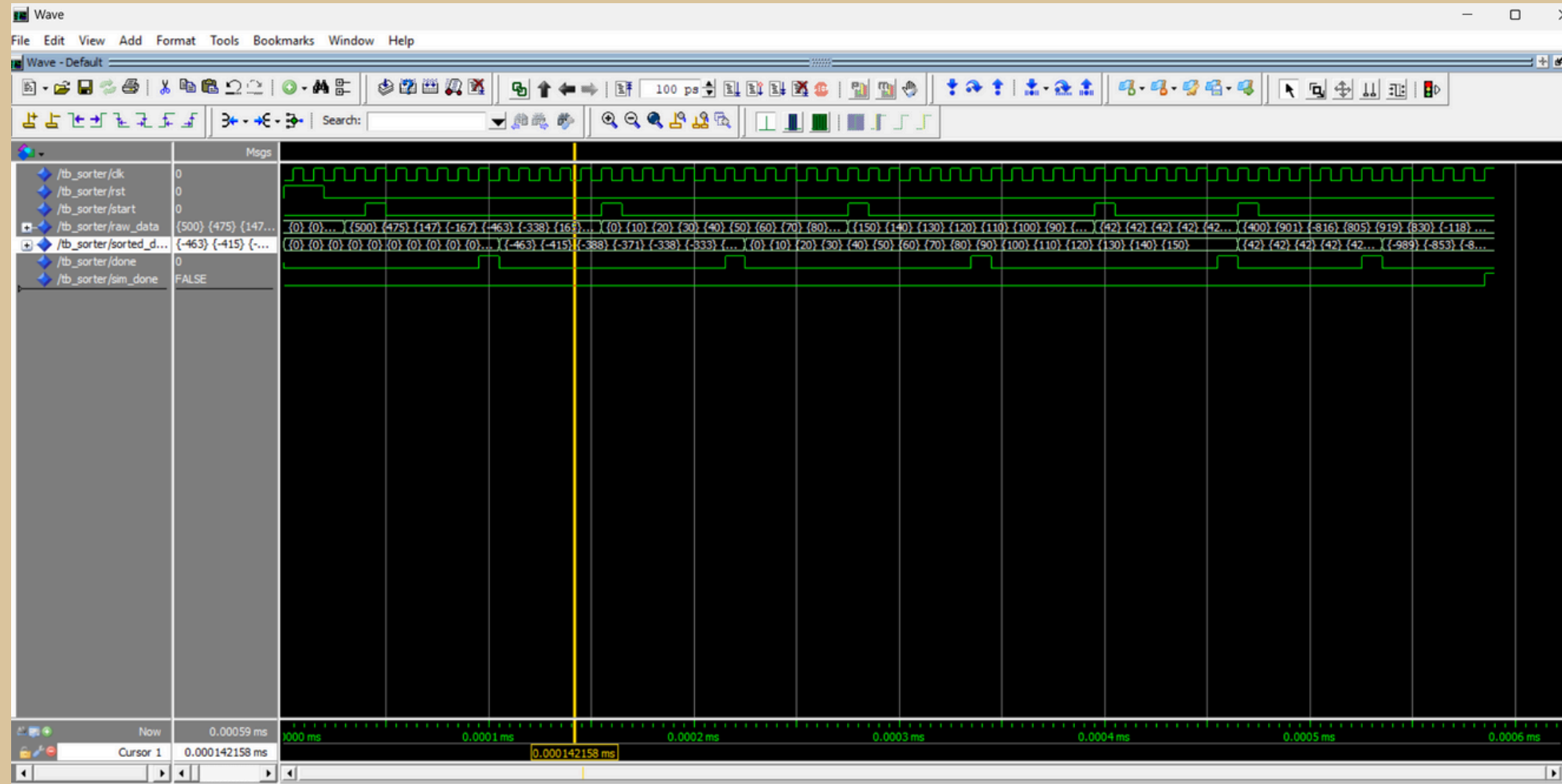
end Behavioral;
```

File ini menginstansiasi top_sorter dan melakukan generate clock 10 ns dan reset awal, lalu menjalankan lima case dimulai dari random, sudah terurut, terbalik, identik, dan bilangan negatif dengan memberi pulse ke sinyal start untuk setiap kasus, setelah top_sorter mengembalikan o_done, testbench membaca o_sorted_data dan memverifikasi urutan ascending menggunakan loop assertion dan mencetak nilai output untuk observasi, sementara fungsi dari paket seperti log_array digunakan untuk logging dan pemeriksaan awal pada test pertama, jika semua pasangan berurutan test memasang laporan "TEST X(Ke berapa) PASSED", kalau tidak akan muncul "SORTING FAILED" dan test dianggap gagal, lalu simulasi berhenti setelah semua test selesai.



RESULT

RESULT



```
# Time: 105 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 1 PASSED ***
# Time: 105 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 155 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 2: Testing with already sorted data...
# Time: 155 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 2 PASSED ***
# Time: 225 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 275 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 3: Testing with reverse sorted data...
# Time: 275 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 3 PASSED ***
# Time: 345 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 395 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 4: Testing with all identical values...
# Time: 395 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 4 PASSED ***
# Time: 465 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 465 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 5: Testing with negative numbers...
# Time: 465 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 5 PASSED ***
# Time: 535 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 585 ns Iteration: 0 Instance: /tb_sorter
# ** Note: ALL TESTS COMPLETED
# Time: 585 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 585 ns Iteration: 0 Instance: /tb_sorter
```

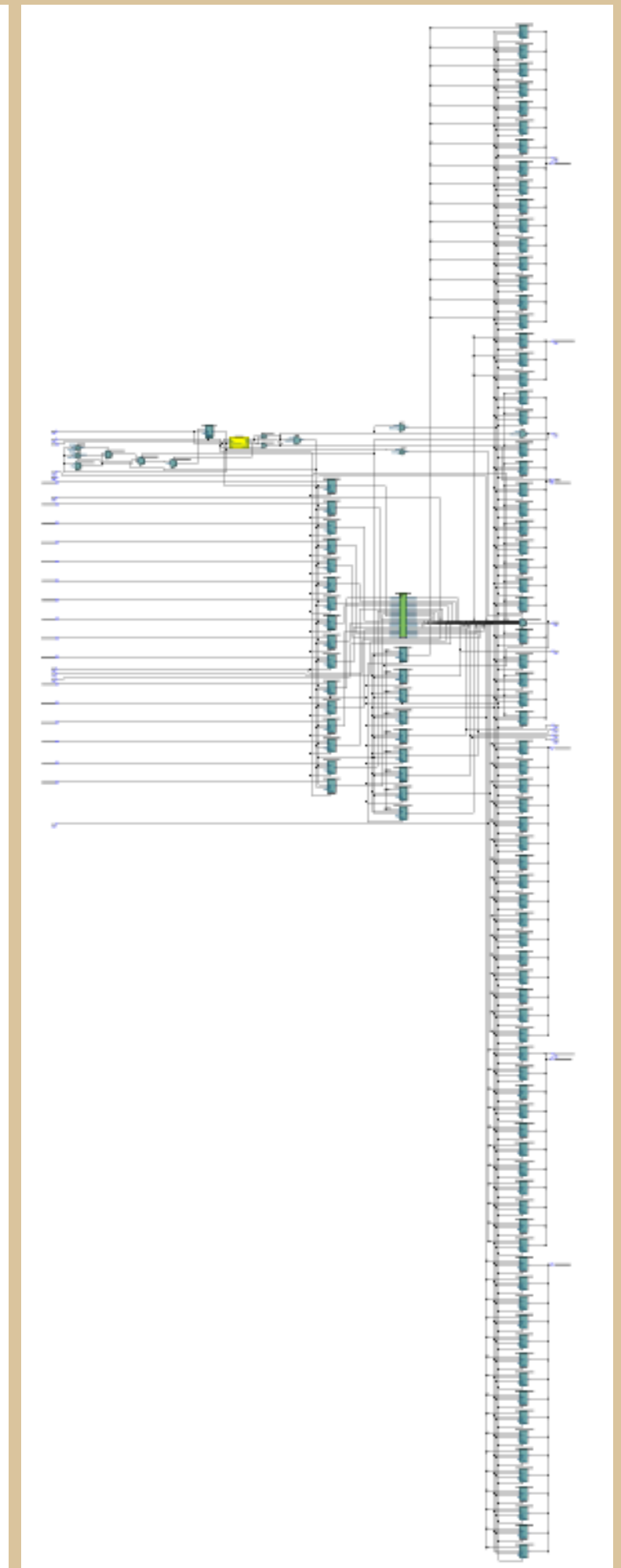
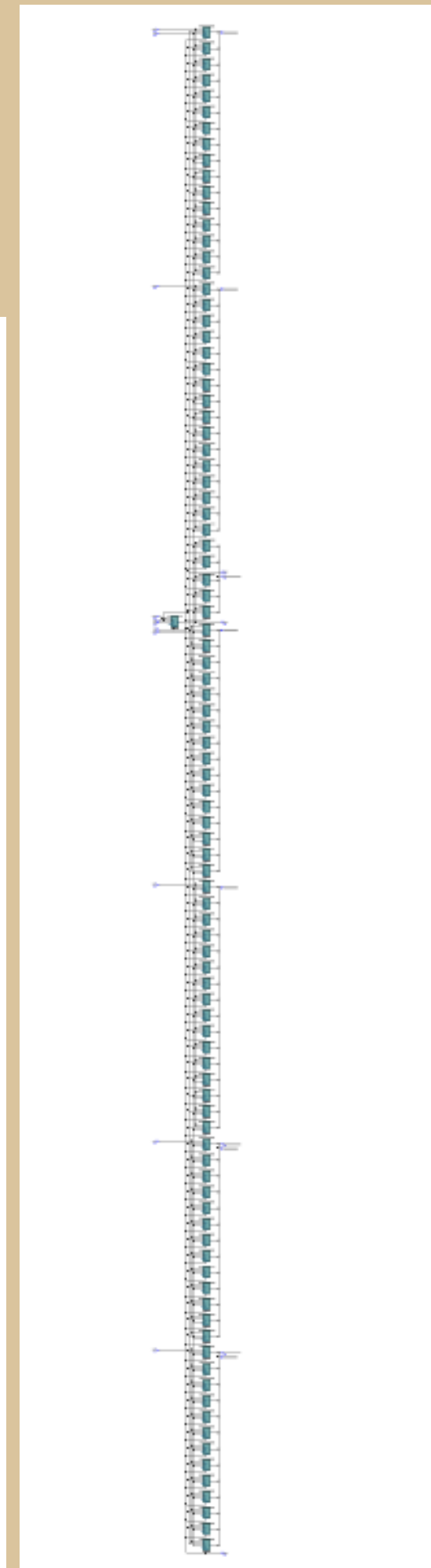
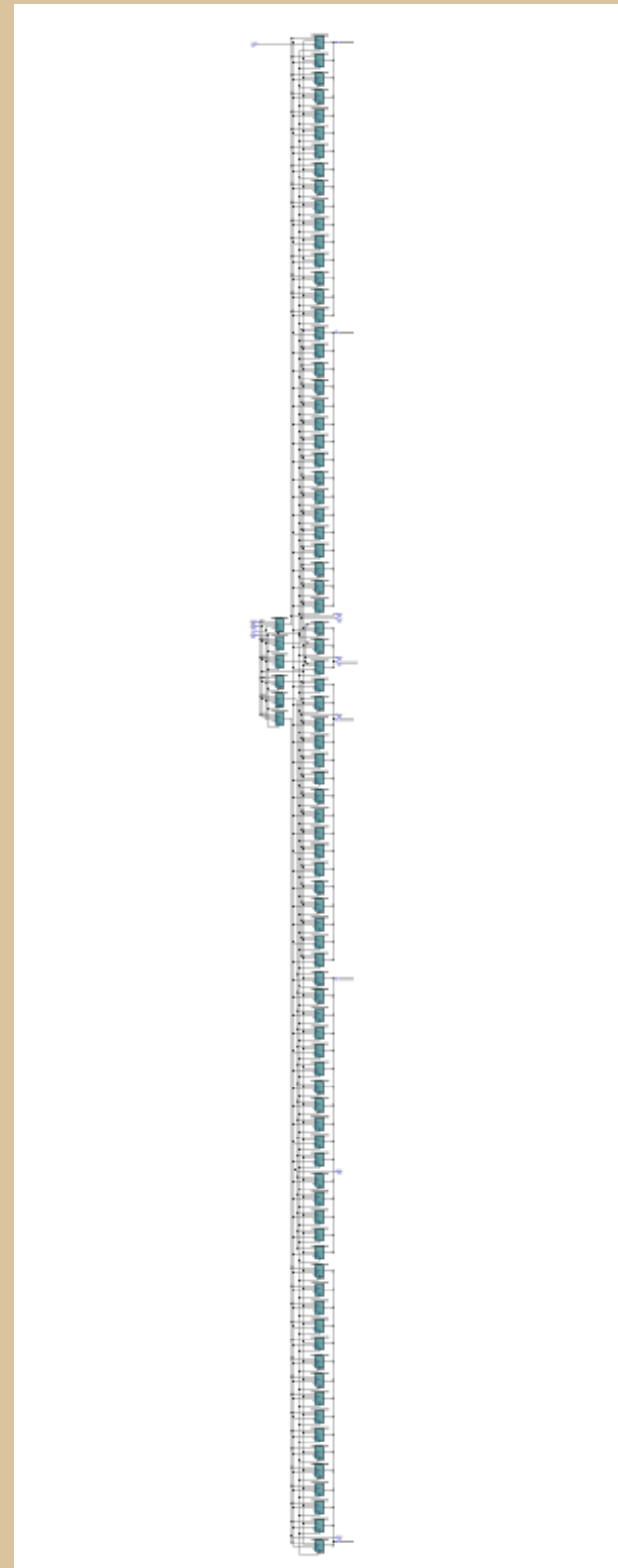


RESULT

RESULT

Hasil sintesis dan simulasi menunjukkan desain sudah bekerja dengan benar dan sesuai secara fungsional, transcript testbench melaporkan semua lima test case “PASSED” dan waveform menunjukkan clock 10 ns, reset/start pulse, o_done pulse, dan nilai o_sorted_data yang stabil setelah beberapa siklus. Tampilan netlist/waveform juga mengonfirmasi jaringan bitonic 10-stage yang tersusun dari banyak unit CAS yang membentuk rantai kombinasional cukup panjang dengan beberapa cluster lokal, sehingga ukuran dan kedalaman logika sesuai dengan tujuan dari implementasi. Pada desain RTL yang pada gambar di samping merupakan implementasi struktural dari bitonic sorter 16-elemen yang sepenuhnya kombinasional (10 stage) dengan banyak instance CAS dan lebar data 16×16 bit, sehingga jika dilihat pada netlist terlihat sangat panjang dan topologi logika sangat dalam, akibatnya fan-out, kapasitas routing, dan kedalaman logika meningkat sehingga critical path menjadi besar dan penggunaan sumber daya juga tinggi. Secara fungsional desain valid dan testbench menunjukkan semua skenario yang sudah berhasil.

-
-
-
-





KESIMPULAN



Final Proyek ini merupakan rancangan sirkuit dari “16-Channel Parallel Sorting Network” berbasis FPGA untuk menggunakan algoritma bitonic sort untuk mengatasi keterbatasan latensi pada CPU konvensional. Sistem yang kami buat ini memanfaatkan VHDL yang menggunakan pendekatan structural model untuk membuat sirkuit yang lebih kompleks, dengan menyusun unit Compare-and-Swap. Hasil dari testbench menunjukkan desain yang dibuat valid untuk menyelesaikan fungsinya dalam mengurutkan data data yang beragam, seperti data acak, terurut terbalik, hingga negatif dengan waktu eksekusi yang sangat cepat.



THANK YOU!



[KELOMPOK10_FINPRO_PSD](#)

